

```

1.

from flask import Blueprint, request, jsonify
from backend.database.db import db
from backend.utils.decorators import require_auth
from backend.services.streak_service import update_user_streak
# FIXED IMPORT - use utils wrapper that handles days_ahead
from backend.utils.notifications import (
    create_follow_up_notification,
    create_motivation_notification
)
from datetime import date, timedelta
import random

applications_bp = Blueprint('applications', __name__)

VALID_STATUSES = ['Planned', 'Applied', 'Interview', 'Offer', 'Rejected']

REJECTION_MESSAGES = [
    "Every 'no' brings you closer to your 'yes'. This rejection is just one step in your journey. Keep going! 🍀",
    "Remember: companies reject candidates for many reasons that have nothing to do with your skills. Stay confident! ☀️",
    "The right opportunity is still out there. This just wasn't the right match. Keep pushing forward! 🔥",
    "Rejection is redirection. Sometimes the universe is protecting you from opportunities that aren't meant for you. ✨",
    "You've overcome challenges before, and you'll overcome this too. Use this as motivation to come back stronger! 💪"
]

# =====
# GET ALL APPLICATIONS
# =====
@applications_bp.route('/', methods=['GET'])
@require_auth
def get_applications():
    try:
        apps = db.query(
            """
            SELECT a.*, c.name AS company_name, c.website AS company_website,
                   c.location AS company_location, c.industry AS company_industry
            FROM applications a
            JOIN companies c ON a.company_id = c.id
            WHERE a.user_id = ?
            ORDER BY a.created_at DESC
            """
            (request.user_id,)
        )

        grouped = {status: [] for status in VALID_STATUSES}
        for app in apps:
            if app['status'] in grouped:
                grouped[app['status']].append(app)

        return jsonify({
            "applications": apps,
            "grouped_by_status": grouped
        }), 200

    except Exception as e:
        return jsonify({"error": str(e)}), 500

# =====
# CREATE APPLICATION
# =====
@applications_bp.route('/', methods=['POST'])
@require_auth
def create_application():
    try:
        data = request.get_json()

        if not data:
            return jsonify({"error": "No data provided"}), 400

        if not data.get('company_id'):
            return jsonify({"error": "company_id is required"}), 400
        if not data.get('job_title'):
            return jsonify({"error": "job_title is required"}), 400
        if not data.get('status'):
            return jsonify({"error": "status is required"}), 400

        company_id = data['company_id']
        job_title = data['job_title'].strip()
        job_url = data.get('job_url', '').strip() or None
        status = data['status']
        notes = data.get('notes', '').strip() or None

        if status not in VALID_STATUSES:
            return jsonify({"error": f"Invalid status. Must be one of: {'', ' '.join(VALID_STATUSES)}"}), 400

        company = db.query_one(
            "SELECT id FROM companies WHERE id = ? AND user_id = ?",
            (company_id, request.user_id)
        )
        if not company:
            return jsonify({"error": "Company not found or doesn't belong to you"}), 400

        applied_date = date.today().isoformat() if status == 'Applied' else None

        db.execute(
            """
            INSERT INTO applications
            (user_id, company_id, job_title, job_url, status, notes, applied_date, created_at)
            VALUES (?, ?, ?, ?, ?, ?, ?, datetime('now'))
            """
            (request.user_id, company_id, job_title, job_url, status, notes, applied_date)
        )
        db.commit()

        app_row = db.query_one(
            """
            SELECT a.*, c.name AS company_name, c.website AS company_website
            FROM applications a
            JOIN companies c ON a.company_id = c.id
            WHERE a.user_id = ?
            ORDER BY a.id DESC
            LIMIT 1
            """
            (request.user_id,)
        )

        if status == 'Applied':
            today = date.today()
            week_start = today - timedelta(days=today.weekday())

            db.execute(

```

```

        """
        UPDATE goals
        SET applications_current = applications_current + 1,
            updated_at = date('now')
        WHERE user_id = ? AND week_start = ?
        """
        (request.user_id, week_start.isoformat())
    )
    db.commit()

    # FIXED: Now using wrapper function that handles days_ahead
    create_follow_up_notification(request.user_id, app_row['id'], days_ahead=7)
    update_user_streak(request.user_id, points=10)

    return jsonify(app_row), 201

except Exception as e:
    return jsonify({"error": str(e)}), 500

# =====
# GET SINGLE APPLICATION
# =====
@applications_bp.route('/<int:app_id>', methods=['GET'])
@require_auth
def get_application(app_id):
    try:
        app_row = db.query_one(
            """
            SELECT a.*, c.name AS company_name, c.website AS company_website,
                c.location AS company_location, c.industry AS company_industry
            FROM applications a
            JOIN companies c ON a.company_id = c.id
            WHERE a.id = ? AND a.user_id = ?
            """
            (app_id, request.user_id)
        )

    if not app_row:
        return jsonify({"error": "Application not found"}), 404

    outreach = db.query(
        """
        SELECT o.*, c.name AS contact_name, c.email AS contact_email
        FROM outreach_activities o
        LEFT JOIN contacts c ON o.contact_id = c.id
        WHERE o.application_id = ? AND o.user_id = ?
        ORDER BY o.sent_date DESC
        """
        (app_id, request.user_id)
    )

    analyses = db.query(
        """
        SELECT id, ats_score, created_at, cv_filename
        FROM cv_analyses
        WHERE application_id = ? AND user_id = ?
        ORDER BY created_at DESC
        """
        (app_id, request.user_id)
    )

    return jsonify({
        "application": app_row,
        "outreach": outreach,
        "cv_analyses": analyses,
    }), 200

except Exception as e:
    return jsonify({"error": str(e)}), 500

# =====
# UPDATE APPLICATION
# =====
@applications_bp.route('/<int:app_id>', methods=['PUT'])
@require_auth
def update_application(app_id):
    try:
        data = request.get_json()
        if not data:
            return jsonify({"error": "No data provided"}), 400

        app_row = db.query_one(
            "SELECT * FROM applications WHERE id = ? AND user_id = ?",
            (app_id, request.user_id)
        )

    if not app_row:
        return jsonify({"error": "Application not found"}), 404

    previous_status = app_row['status']

    updates = []
    params = []

    # Store the new status if provided for later use
    new_status = None

    if 'job_title' in data and data['job_title']:
        updates.append("job_title = ?")
        params.append(data['job_title'].strip())

    if 'job_url' in data:
        updates.append("job_url = ?")
        params.append(data['job_url'].strip() or None)

    if 'notes' in data:
        updates.append("notes = ?")
        params.append(data['notes'].strip() or None)

    if 'status' in data:
        new_status = data['status']
        if new_status not in VALID_STATUSES:
            return jsonify({"error": f"Invalid status. Must be one of: {', '.join(VALID_STATUSES)}"}), 400
        updates.append("status = ?")
        params.append(new_status)

    # CRITICAL FIX: If changing to 'Applied', set applied_date in the SAME UPDATE
    # This is required because of the database constraint
    if new_status == 'Applied' and previous_status != 'Applied':
        updates.append("applied_date = ?")
        params.append(date.today().isoformat())

    if not updates:

```

```

        return jsonify({"error": "No valid fields to update"}), 400

    # Execute update with all changes in ONE transaction
    params.append(app_id)
    db.execute(
        f"UPDATE applications SET ({', '.join(updates)}) WHERE id = ?",
        tuple(params)
    )
    db.commit()

    # Handle status change side effects AFTER successful update
    if new_status and new_status != previous_status:
        if new_status == 'Applied':
            today = date.today()
            week_start = today - timedelta(days=today.weekday())

            db.execute(
                """
                UPDATE goals
                SET applications_current = applications_current + 1,
                    updated_at = date('now')
                WHERE user_id = ? AND week_start = ?
                """
                (request.user_id, week_start.isoformat())
            )
            db.commit()

            # FIXED: Now using wrapper function that handles days_ahead
            create_follow_up_notification(request.user_id, app_id, days_ahead=7)
            update_user_streak(request.user_id, points=10)

        elif new_status == 'Rejected':
            message = random.choice(REJECTION_MESSAGES)
            create_motivation_notification(request.user_id, message)

    # Fetch and return the updated application
    updated_app = db.query_one(
        """
        SELECT a.*, c.name AS company_name, c.website AS company_website
        FROM applications a
        JOIN companies c ON a.company_id = c.id
        WHERE a.id = ? AND a.user_id = ?
        """
        (app_id, request.user_id)
    )

    return jsonify(updated_app), 200

except Exception as e:
    return jsonify({"error": str(e)}), 500

# =====
# DELETE APPLICATION
# =====
@applications_bp.route('/<int:app_id>', methods=['DELETE'])
@require_auth
def delete_application(app_id):
    try:
        app_row = db.query_one(
            "SELECT id FROM applications WHERE id = ? AND user_id = ?",
            (app_id, request.user_id)
        )

        if not app_row:
            return jsonify({"error": "Application not found"}), 404

        db.execute("DELETE FROM applications WHERE id = ?", (app_id,))
        db.commit()

        return jsonify({"message": "Application deleted successfully"}), 200

    except Exception as e:
        return jsonify({"error": str(e)}), 500

2.
from flask import Blueprint, request, jsonify
from backend.database.db import db
from backend.utils.decorators import require_auth
from backend.utils.helpers import hash_password, verify_password, generate_token
from backend.utils.validators import validate_email
import re

auth_bp = Blueprint('auth', __name__)

def validate_password_strength(password: str) -> tuple[bool, str]:
    """
    Validate password strength.
    """
    if len(password) < 8:
        return False, "Password must be at least 8 characters"
    if not re.search(r'[A-Z]', password):
        return False, "Password must contain at least one uppercase letter"
    if not re.search(r'[a-z]', password):
        return False, "Password must contain at least one lowercase letter"
    if not re.search(r'\d', password):
        return False, "Password must contain at least one number"
    if not re.search(r'!@#$%^&*()_~\`{}|<>|', password):
        return False, "Password must contain at least one Special character"
    return True, ""

# =====
# REGISTER
# =====
@app_bp.route('/register', methods=['POST'])
def register():
    try:
        data = request.get_json()

        if not data or not data.get("email") or not data.get("password") or not data.get("name"):
            return jsonify({"error": "Email, password, and name are required"}), 400

        email = data["email"].strip().lower()
        password = data["password"]
        name = data["name"].strip()

        # Validate email format
        if not validate_email(email):
            return jsonify({"error": "Invalid email format"}), 400

```

```

# Validate password
valid, msg = validate_password_strength(password)
if not valid:
    return jsonify({"error": msg}), 400

# Check duplicate user
exists = db.query_one(
    "SELECT id FROM users WHERE LOWER(email) = LOWER(?)",
    (email,)
)
if exists:
    return jsonify({"error": "User with this email already exists"}), 400

# Hash password
hashed_pw = hash_password(password)

# Insert user (DATABASE TRIGGER will auto-create streak)
db.execute(
    """
    INSERT INTO users (email, password_hash, name, created_at)
    VALUES (?, ?, ?, datetime('now'))
    """,
    (email, hashed_pw, name)
)
db.commit()

# FIXED: Fetch new user with created_at
user = db.query_one(
    "SELECT id, email, name, created_at FROM users WHERE email = ?",
    (email,)
)

if not user:
    return jsonify({"error": "User creation failed"}), 500

# Generate token
token = generate_token(user["id"])

# FIXED: Include created_at in response
return jsonify({
    "user": {
        "id": user["id"],
        "email": user["email"],
        "name": user["name"],
        "created_at": user["created_at"]
    },
    "token": token
}), 201

except Exception as e:
    db.rollback()
    return jsonify({"error": str(e)}), 500

# -----
# LOGIN
# -----
@auth_bp.route('/login', methods=['POST'])
def login():
    try:
        data = request.get_json()

        if not data or not data.get("email") or not data.get("password"):
            return jsonify({"error": "Email and password are required"}), 400

        email = data["email"].strip().lower()
        password = data["password"]

        # Fetch user
        user = db.query_one(
            "SELECT id, email, name, password_hash FROM users WHERE LOWER(email) = LOWER(?)",
            (email,)
        )

        if not user:
            return jsonify({"error": "Invalid email or password"}), 401

        # Verify password
        if not verify_password(password, user["password_hash"]):
            return jsonify({"error": "Invalid email or password"}), 401

        # Update last login
        db.execute(
            "UPDATE users SET last_login = datetime('now') WHERE id = ?",
            (user["id"],)
        )
        db.commit()

        # FIXED: Get updated last_login
        updated_user = db.query_one(
            "SELECT id, email, name, last_login FROM users WHERE id = ?",
            (user["id"],)
        )

        # Check onboarding status
        onboarding_done = db.query_one(
            "SELECT id FROM onboarding_data WHERE user_id = ?",
            (user["id"],)
        )
        is not None

        # Generate token
        token = generate_token(user["id"])

        # FIXED: Include last_login in response
        return jsonify({
            "user": {
                "id": updated_user["id"],
                "email": updated_user["email"],
                "name": updated_user["name"],
                "last_login": updated_user["last_login"]
            },
            "token": token,
            "has_completed_onboarding": onboarding_done
        }), 200

    except Exception as e:
        return jsonify({"error": str(e)}), 500

# -----
# GET PROFILE
# -----
@auth_bp.route('/profile', methods=['GET'])
@require_auth

```

```

def get_profile():
    try:
        user = db.query_one(
            """
            SELECT id, email, name, created_at, last_login, is_active,
                   email_notifications_enabled, notification_preferences
            FROM users
            WHERE id = ?
            """,
            (request.user_id,)
        )

        if not user:
            return jsonify({"error": "User not found"}), 404

        return jsonify(user), 200

    except Exception as e:
        return jsonify({"error": str(e)}), 500

# -----
# UPDATE PROFILE
# -----
@auth_bp.route('/profile', methods=['PUT'])
@require_auth
def update_profile():
    try:
        data = request.get_json()

        if not data:
            return jsonify({"error": "No data provided"}), 400

        user = db.query_one(
            "SELECT id, email FROM users WHERE id = ?",
            (request.user_id,)
        )

        if not user:
            return jsonify({"error": "User not found"}), 404

        updates = []
        params = []

        if "name" in data and data["name"]:
            updates.append("name = ?")
            params.append(data["name"].strip())

        if "email" in data and data["email"]:
            new_email = data["email"].strip().lower()

            if not validate_email(new_email):
                return jsonify({"error": "Invalid email format"}), 400

            # Check if email taken
            exists = db.query_one(
                "SELECT id FROM users WHERE LOWER(email) = LOWER(?) AND id != ?",
                (new_email, request.user_id)
            )
            if exists:
                return jsonify({"error": "Email already in use"}), 400

            updates.append("email = ?")
            params.append(new_email)

        if "email_notifications_enabled" in data:
            updates.append("email_notifications_enabled = ?")
            params.append(1 if data["email_notifications_enabled"] else 0)

        if not updates:
            return jsonify({"error": "No valid fields to update"}), 400

        params.append(request.user_id)

        db.execute(
            f"UPDATE users SET {'', '.join(updates)} WHERE id = ?",
            tuple(params)
        )
        db.commit()

        updated = db.query_one(
            """
            SELECT id, email, name, created_at, last_login, is_active,
                   email_notifications_enabled, notification_preferences
            FROM users WHERE id = ?
            """,
            (request.user_id,)
        )

        return jsonify(updated), 200

    except Exception as e:
        db.rollback()
        return jsonify({"error": str(e)}), 500

# -----
# DELETE PROFILE
# -----
@auth_bp.route('/profile', methods=['DELETE'])
@require_auth
def delete_profile():
    try:
        user = db.query_one(
            "SELECT id FROM users WHERE id = ?",
            (request.user_id,)
        )

        if not user:
            return jsonify({"error": "User not found"}), 404

        db.execute(
            "DELETE FROM users WHERE id = ?",
            (request.user_id,)
        )
        db.commit()

        return jsonify({"message": "Account deleted successfully"}), 200

    except Exception as e:
        db.rollback()
        return jsonify({"error": str(e)}), 500

```

```

3.
from flask import Blueprint, request, jsonify
from backend.database.db import db
from backend.utils.decorators import require_auth
from backend.utils.helpers import hash_password, verify_password, generate_token
from backend.utils.validators import validate_email
import re

auth_bp = Blueprint('auth', __name__)

def validate_password_strength(password: str) -> tuple[bool, str]:
    """
    Validate password strength.
    """
    if len(password) < 8:
        return False, "Password must be at least 8 characters"
    if not re.search(r'[A-Z]', password):
        return False, "Password must contain at least one uppercase letter"
    if not re.search(r'[a-z]', password):
        return False, "Password must contain at least one lowercase letter"
    if not re.search(r'\d', password):
        return False, "Password must contain at least one number"
    if not re.search(r'!@#$%^&*().,?\"{}|<>|', password):
        return False, "Password must contain at least one special character"
    return True, ""

# =====
# REGISTER
# =====
@auth_bp.route('/register', methods=['POST'])
def register():
    try:
        data = request.get_json()

        if not data or not data.get("email") or not data.get("password") or not data.get("name"):
            return jsonify({"error": "Email, password, and name are required"}), 400

        email = data["email"].strip().lower()
        password = data["password"]
        name = data["name"].strip()

        # Validate email format
        if not validate_email(email):
            return jsonify({"error": "Invalid email format"}), 400

        # Validate password
        valid, msg = validate_password_strength(password)
        if not valid:
            return jsonify({"error": msg}), 400

        # Check duplicate user
        exists = db.query_one(
            "SELECT id FROM users WHERE LOWER(email) = LOWER(?)",
            (email,)
        )
        if exists:
            return jsonify({"error": "User with this email already exists"}), 400

        # Hash password
        hashed_pw = hash_password(password)

        # Insert user (DATABASE TRIGGER will auto-create streak)
        db.execute(
            """
            INSERT INTO users (email, password_hash, name, created_at)
            VALUES (?, ?, ?, datetime('now'))
            """
            (email, hashed_pw, name)
        )
        db.commit()

        # FIXED: Fetch new user with created_at
        user = db.query_one(
            "SELECT id, email, name, created_at FROM users WHERE email = ?",
            (email,)
        )

        if not user:
            return jsonify({"error": "User creation failed"}), 500

        # Generate token
        token = generate_token(user["id"])

        # FIXED: Include created_at in response
        return jsonify({
            "user": {
                "id": user["id"],
                "email": user["email"],
                "name": user["name"],
                "created_at": user["created_at"]
            },
            "token": token
        }), 201

    except Exception as e:
        db.rollback()
        return jsonify({"error": str(e)}), 500

# =====
# LOGIN
# =====
@auth_bp.route('/login', methods=['POST'])
def login():
    try:
        data = request.get_json()

        if not data or not data.get("email") or not data.get("password"):
            return jsonify({"error": "Email and password are required"}), 400

        email = data["email"].strip().lower()
        password = data["password"]

        # Fetch user
        user = db.query_one(
            "SELECT id, email, name, password_hash FROM users WHERE LOWER(email) = LOWER(?)",
            (email,)
        )

        if not user:
            return jsonify({"error": "Invalid email or password"}), 401
    
```

```

# Verify password
if not verify_password(password, user["password_hash"]):
    return jsonify({"error": "Invalid email or password"}), 401

# Update last login
db.execute(
    "UPDATE users SET last_login = datetime('now') WHERE id = ?",
    (user["id"],)
)
db.commit()

# FIXED: Get updated last_login
updated_user = db.query_one(
    "SELECT id, email, name, last_login FROM users WHERE id = ?",
    (user["id"],)
)

# Check onboarding status
onboarding_done = db.query_one(
    "SELECT id FROM onboarding_data WHERE user_id = ?",
    (user["id"],)
) is not None

# Generate token
token = generate_token(user["id"])

# FIXED: Include last_login in response
return jsonify({
    "user": {
        "id": updated_user["id"],
        "email": updated_user["email"],
        "name": updated_user["name"],
        "last_login": updated_user["last_login"]
    },
    "token": token,
    "has_completed_onboarding": onboarding_done
}), 200

except Exception as e:
    return jsonify({"error": str(e)}), 500

# =====
# GET PROFILE
# =====
@auth_bp.route('/profile', methods=['GET'])
@require_auth
def get_profile():
    try:
        user = db.query_one(
            """
            SELECT id, email, name, created_at, last_login, is_active,
                   email_notifications_enabled, notification_preferences
            FROM users
            WHERE id = ?
            """
            (request.user_id,)
        )

        if not user:
            return jsonify({"error": "User not found"}), 404

        return jsonify(user), 200

    except Exception as e:
        return jsonify({"error": str(e)}), 500

# =====
# UPDATE PROFILE
# =====
@auth_bp.route('/profile', methods=['PUT'])
@require_auth
def update_profile():
    try:
        data = request.get_json()

        if not data:
            return jsonify({"error": "No data provided"}), 400

        user = db.query_one(
            "SELECT id, email FROM users WHERE id = ?",
            (request.user_id,)
        )

        if not user:
            return jsonify({"error": "User not found"}), 404

        updates = []
        params = []

        if "name" in data and data["name"]:
            updates.append("name = ?")
            params.append(data["name"].strip())

        if "email" in data and data["email"]:
            new_email = data["email"].strip().lower()

            if not validate_email(new_email):
                return jsonify({"error": "Invalid email format"}), 400

            # Check if email taken
            exists = db.query_one(
                "SELECT id FROM users WHERE LOWER(email) = LOWER(?) AND id != ?",
                (new_email, request.user_id)
            )
            if exists:
                return jsonify({"error": "Email already in use"}), 400

            updates.append("email = ?")
            params.append(new_email)

        if "email_notifications_enabled" in data:
            updates.append("email_notifications_enabled = ?")
            params.append(1 if data["email_notifications_enabled"] else 0)

        if not updates:
            return jsonify({"error": "No valid fields to update"}), 400

        params.append(request.user_id)

        db.execute(
            f"UPDATE users SET {', '.join(updates)} WHERE id = ?",
            tuple(params)
        )

```

```

    )
    db.commit()

    updated = db.query_one(
        """
        SELECT id, email, name, created_at, last_login, is_active,
               email_notifications_enabled, notification_preferences
        FROM users WHERE id = ?
        """,
        (request.user_id,)
    )

    return jsonify(updated), 200

except Exception as e:
    db.rollback()
    return jsonify({"error": str(e)}), 500

# -----
# DELETE PROFILE
# -----
@auth_bp.route('/profile', methods=['DELETE'])
@require_auth
def delete_profile():
    try:
        user = db.query_one(
            "SELECT id FROM users WHERE id = ?",
            (request.user_id,)
        )

        if not user:
            return jsonify({"error": "User not found"}), 404

        db.execute(
            "DELETE FROM users WHERE id = ?",
            (request.user_id,)
        )
        db.commit()

        return jsonify({"message": "Account deleted successfully"}), 200

    except Exception as e:
        db.rollback()
        return jsonify({"error": str(e)}), 500

4.
from flask import Blueprint, request, jsonify
from backend.database.db import db
from backend.utils.decorators import require_auth

companies_bp = Blueprint('companies', __name__)

@companies_bp.route('/', methods=['GET'])
@require_auth
def get_companies():
    """
    GET /api/v1/companies
    Query: ?industry=Technology
    """
    try:
        industry = request.args.get('industry')

        if industry:
            companies = db.query(
                """
                SELECT * FROM companies
                WHERE user_id = ? AND industry = ?
                ORDER BY created_at DESC
                """,
                (request.user_id, industry)
            )
        else:
            companies = db.query(
                """
                SELECT * FROM companies
                WHERE user_id = ?
                ORDER BY created_at DESC
                """,
                (request.user_id,)
            )

        return jsonify(companies), 200

    except Exception as e:
        return jsonify({"error": str(e)}), 500

@companies_bp.route('/', methods=['POST'])
@require_auth
def create_company():
    """
    POST /api/v1/companies
    Create company
    """
    try:
        data = request.get_json()

        if not data or not data.get('name'):
            return jsonify({"error": "Company name is required"}), 400

        name = data['name'].strip()
        website = data.get('website', '').strip() or None
        location = data.get('location', '').strip() or None
        industry = data.get('industry', '').strip() or None
        notes = data.get('notes', '').strip() or None

        if len(name) < 2:
            return jsonify({"error": "Company name must be at least 2 characters"}), 400

        # Prevent duplicate names
        existing = db.query_one(
            """
            SELECT id FROM companies
            WHERE user_id = ? AND LOWER(name) = LOWER(?)
            """,
            (request.user_id, name)
        )

        if existing:
            return jsonify({"error": "Company with this name already exists"}), 400

        db.execute(
            """
            INSERT INTO companies (user_id, name, website, location, industry, notes, created_at)
            VALUES (?, ?, ?, ?, ?, ?, date('now'))
            """

```



```

        """
        (request.user_id, name, website, location, industry, notes)
    )
    db.commit()

    company = db.query_one(
        """
        SELECT * FROM companies
        WHERE user_id = ?
        ORDER BY id DESC
        LIMIT 1
        """,
        (request.user_id,)
    )

    return jsonify(company), 201

except Exception as e:
    return jsonify({"error": str(e)}), 500

@companies_bp.route('/<int:company_id>', methods=['GET'])
@require_auth
def get_company(company_id):
    """
    GET /api/v1/companies/<id>
    Returns company + applications + contacts
    """
    try:
        company = db.query_one(
            "SELECT * FROM companies WHERE id = ? AND user_id = ?",
            (company_id, request.user_id)
        )

        if not company:
            return jsonify({"error": "Company not found"}), 404

        applications = db.query(
            """
            SELECT id, job_title, status, applied_date, created_at
            FROM applications
            WHERE company_id = ? AND user_id = ?
            ORDER BY created_at DESC
            """,
            (company_id, request.user_id)
        )

        contacts = db.query(
            """
            SELECT * FROM contacts
            WHERE company_id = ?
            ORDER BY name ASC
            """,
            (company_id,)
        )

        return jsonify({
            "company": company,
            "applications": applications,
            "contacts": contacts
        }), 200

    except Exception as e:
        return jsonify({"error": str(e)}), 500

@companies_bp.route('/<int:company_id>', methods=['PUT'])
@require_auth
def update_company(company_id):
    """
    Update company fields
    """
    try:
        data = request.get_json()

        if not data:
            return jsonify({"error": "No data provided"}), 400

        company = db.query_one(
            "SELECT * FROM companies WHERE id = ? AND user_id = ?",
            (company_id, request.user_id)
        )

        if not company:
            return jsonify({"error": "Company not found"}), 404

        updates = []
        params = []

        if 'name' in data and data['name']:
            name = data['name'].strip()
            if len(name) < 2:
                return jsonify({"error": "Company name must be at least 2 characters"}), 400

            existing = db.query_one(
                """
                SELECT id FROM companies
                WHERE user_id = ? AND LOWER(name) = LOWER(?) AND id != ?
                """,
                (request.user_id, name, company_id)
            )
            if existing:
                return jsonify({"error": "Company with this name already exists"}), 400

            updates.append("name = ?")
            params.append(name)

        if 'website' in data:
            updates.append("website = ?")
            params.append(data['website'].strip() or None)

        if 'location' in data:
            updates.append("location = ?")
            params.append(data['location'].strip() or None)

        if 'industry' in data:
            updates.append("industry = ?")
            params.append(data['industry'].strip() or None)

        if 'notes' in data:
            updates.append("notes = ?")
            params.append(data['notes'].strip() or None)

        if not updates:
            return jsonify({"error": "No valid fields to update"}), 400

```

```

        params.append(company_id)

        db.execute(
            f"UPDATE companies SET ('.join(updates)) WHERE id = ?",
            tuple(params)
        )
        db.commit()

        updated_company = db.query_one(
            "SELECT * FROM companies WHERE id = ?",
            (company_id,)
        )

        return jsonify(updated_company), 200

    except Exception as e:
        return jsonify({"error": str(e)}), 500

@companies_bp.route('/<int:company_id>', methods=['DELETE'])
@require_auth
def delete_company(company_id):
    """
    Delete company (cascade removes contacts + applications)
    """
    try:
        company = db.query_one(
            "SELECT id FROM companies WHERE id = ? AND user_id = ?",
            (company_id, request.user_id)
        )

        if not company:
            return jsonify({"error": "Company not found"}), 404

        db.execute("DELETE FROM companies WHERE id = ?", (company_id,))
        db.commit()

        return jsonify({"message": "Company deleted successfully"}), 200

    except Exception as e:
        return jsonify({"error": str(e)}), 500

5.from flask import Blueprint, request, jsonify
from backend.database.db import db
from backend.utils.decorators import require_auth
from backend.utils.validators import validate_email

contacts_bp = Blueprint('contacts', __name__)

@contacts_bp.route('', methods=['GET'])
@require_auth
def get_contacts():
    """
    GET /api/v1/contacts?company_id=X
    """
    try:
        company_id = request.args.get('company_id')
        if not company_id:
            return jsonify({"error": "company_id query parameter is required"}), 400

        company = db.query_one(
            "SELECT id FROM companies WHERE id = ? AND user_id = ?",
            (company_id, request.user_id)
        )

        if not company:
            return jsonify({"error": "Company not found"}), 404

        contacts = db.query(
            """
            SELECT * FROM contacts
            WHERE company_id = ?
            ORDER BY name ASC
            """,
            (company_id,)
        )

        return jsonify(contacts), 200

    except Exception as e:
        return jsonify({"error": str(e)}), 500

@contacts_bp.route('', methods=['POST'])
@require_auth
def create_contact():
    """
    Create a new contact for a company
    """
    try:
        data = request.get_json()

        if not data or not data.get('company_id') or not data.get('name'):
            return jsonify({"error": "company_id and name are required"}), 400

        company_id = data['company_id']
        name = data['name'].strip()
        role = data.get('role', '').strip() or None
        email = data.get('email', '').strip() or None
        linkedin_url = data.get('linkedin_url', '').strip() or None
        notes = data.get('notes', '').strip() or None

        company = db.query_one(
            "SELECT id FROM companies WHERE id = ? AND user_id = ?",
            (company_id, request.user_id)
        )
        if not company:
            return jsonify({"error": "Company not found"}), 404

        if email and not validate_email(email):
            return jsonify({"error": "Invalid email format"}), 400

        if email:
            existing = db.query_one(
                """
                SELECT id FROM contacts
                WHERE company_id = ? AND LOWER(email) = LOWER(?)
                """,
                (company_id, email)
            )
            if existing:
                return jsonify({"error": "Contact with this email already exists for this company"}), 400

```

```

        db.execute(
            """
            INSERT INTO contacts (company_id, name, role, email, linkedin_url, notes, created_at)
            VALUES (?, ?, ?, ?, ?, ?, date('now'))
            """
            (company_id, name, role, email, linkedin_url, notes)
        )
        db.commit()

        contact = db.query_one(
            """
            SELECT * FROM contacts
            WHERE company_id = ?
            ORDER BY id DESC
            LIMIT 1
            """
            (company_id,)
        )

        return jsonify(contact), 201

    except Exception as e:
        return jsonify({"error": str(e)}), 500

@contacts_bp.route('/<int:contact_id>', methods=['PUT'])
@require_auth
def update_contact(contact_id):
    """
    Update a contact
    """
    try:
        data = request.get_json()
        if not data:
            return jsonify({"error": "No data provided"}), 400

        contact = db.query_one(
            """
            SELECT c.*, comp.user_id
            FROM contacts c
            JOIN companies comp ON c.company_id = comp.id
            WHERE c.id = ?
            """
            (contact_id,)
        )

        if not contact:
            return jsonify({"error": "Contact not found"}), 404

        if contact['user_id'] != request.user_id:
            return jsonify({"error": "Unauthorized"}), 403

        updates = []
        params = []

        if 'name' in data and data['name']:
            updates.append("name = ?")
            params.append(data['name'].strip())

        if 'role' in data:
            updates.append("role = ?")
            params.append(data['role'].strip() or None)

        if 'email' in data:
            email = data['email'].strip() or None
            if email and not validate_email(email):
                return jsonify({"error": "Invalid email format"}), 400

            if email:
                existing = db.query_one(
                    """
                    SELECT id FROM contacts
                    WHERE company_id = ? AND LOWER(email) = LOWER(?) AND id != ?
                    """
                    (contact['company_id'], email, contact_id)
                )
                if existing:
                    return jsonify({"error": "Contact with this email already exists"}), 400

            updates.append("email = ?")
            params.append(email)

        if 'linkedin_url' in data:
            updates.append("linkedin_url = ?")
            params.append(data['linkedin_url'].strip() or None)

        if 'notes' in data:
            updates.append("notes = ?")
            params.append(data['notes'].strip() or None)

        if not updates:
            return jsonify({"error": "No valid fields to update"}), 400

        params.append(contact_id)

        db.execute(
            f"UPDATE contacts SET {' '.join(updates)} WHERE id = ?",
            tuple(params)
        )
        db.commit()

        updated_contact = db.query_one(
            "SELECT * FROM contacts WHERE id = ?",
            (contact_id,)
        )

        return jsonify(updated_contact), 200

    except Exception as e:
        return jsonify({"error": str(e)}), 500

@contacts_bp.route('/<int:contact_id>', methods=['DELETE'])
@require_auth
def delete_contact(contact_id):
    """
    Delete a contact
    """
    try:
        contact = db.query_one(
            """
            SELECT c.id, comp.user_id
            FROM contacts c
            JOIN companies comp ON c.company_id = comp.id

```

```

        WHERE c.id = ?
        """
        (contact_id,)
    )

    if not contact:
        return jsonify({"error": "Contact not found"}), 404

    if contact['user_id'] != request.user_id:
        return jsonify({"error": "Unauthorized"}), 403

    db.execute("DELETE FROM contacts WHERE id = ?", (contact_id,))
    db.commit()

    return jsonify({"message": "Contact deleted successfully"}), 200

except Exception as e:
    return jsonify({"error": str(e)}), 500

6.
from flask import Blueprint, request, jsonify, current_app
from backend.database.db import db
from backend.utils.decorators import require_auth
from werkzeug.utils import secure_filename
import os
import PyPDF2
import docx
import json
import re
from datetime import datetime

cv_matcher_bp = Blueprint('cv_matcher', __name__)

ALLOWED_EXTENSIONS = {'pdf', 'docx', 'doc'}

def allowed_file(filename):
    """Check if file extension is allowed"""
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS

def extract_text_from_pdf(file_path):
    """Extract text from PDF file"""
    try:
        text = ""
        with open(file_path, 'rb') as file:
            pdf_reader = PyPDF2.PdfReader(file)
            for page in pdf_reader.pages:
                text += page.extract_text() + "\n"
        return text
    except Exception as e:
        raise Exception(f"Error extracting text from PDF: {str(e)}")

def extract_text_from_docx(file_path):
    """Extract text from DOCX file"""
    try:
        doc = docx.Document(file_path)
        text = "\n".join([paragraph.text for paragraph in doc.paragraphs])
        return text
    except Exception as e:
        raise Exception(f"Error extracting text from DOCX: {str(e)}")

def normalize_text(text):
    """Normalize text: lowercase, remove extra spaces and punctuation"""
    text = text.lower()
    text = re.sub(r'[^\w\s]', ' ', text)
    text = re.sub(r'\s+', ' ', text)
    return text.strip()

def extract_keywords(text):
    """Extract skills and keywords from text"""
    # Common technical skills
    technical_skills = [
        'python', 'java', 'javascript', 'typescript', 'c++', 'c#', 'ruby', 'php', 'swift', 'kotlin',
        'react', 'angular', 'vue', 'node.js', 'nodejs', 'express', 'django', 'flask', 'spring',
        'docker', 'kubernetes', 'aws', 'azure', 'gcp', 'terraform', 'jenkins', 'git', 'github',
        'sql', 'nosql', 'mongodb', 'postgresql', 'mysql', 'redis', 'elasticsearch',
        'machine learning', 'deep learning', 'ai', 'data science', 'tensorflow', 'pytorch',
        'rest api', 'graphql', 'microservices', 'agile', 'scrum', 'ci/cd', 'devops',
        'html', 'css', 'sass', 'webpack', 'babel', 'npm', 'yarn',
        'unit testing', 'integration testing', 'test driven development', 'tdd'
    ]

    # Soft skills
    soft_skills = [
        'communication', 'leadership', 'teamwork', 'problem solving', 'analytical',
        'critical thinking', 'time management', 'collaboration', 'adaptability',
        'presentation', 'mentoring', 'project management'
    ]

    all_skills = technical_skills + soft_skills
    normalized_text = normalize_text(text)

    found_keywords = []
    for skill in all_skills:
        if skill in normalized_text:
            found_keywords.append(skill)

    return list(set(found_keywords)) # Remove duplicates

def calculate_ats_score(cv_keywords, jd_keywords):
    """Calculate ATS score based on keyword matching"""
    if not jd_keywords:
        return 0

    matched = set(cv_keywords) & set(jd_keywords)
    score = (len(matched) / len(jd_keywords)) * 100
    return min(int(score), 100) # Cap at 100

def generate_suggestions(matched_keywords, missing_keywords):
    """Generate improvement suggestions"""
    suggestions = []

    if len(missing_keywords) > 0:
        suggestions.append({
            "category": "Missing Keywords",
            "suggestion": f"Add these keywords to your CV: {' '.join(missing_keywords[:5])}"
        })

    if len(matched_keywords) < 5:
        suggestions.append({
            "category": "Keyword Density",
            "suggestion": "Include more relevant technical skills and tools mentioned in the job description."
        })

    suggestions.append({

```

```

        "category": "Formatting",
        "suggestion": "Use clear section headers like 'Skills', 'Experience', 'Education' to help ATS systems parse your CV."
    })

    suggestions.append({
        "category": "Quantify Achievements",
        "suggestion": "Add metrics and numbers to your achievements (e.g., 'Improved performance by 40%')."
    })

    return suggestions

@cv_matcher_bp.route('/analyze', methods=['POST'])
@require_auth
def analyze_cv():
    """
    POST /api/v1/cv/analyze

    Analyze CV against job description (protected)

    Form data:
    - cv_file: PDF or DOCX file (required)
    - jd_text: Job description text (optional if jd_file provided)
    - jd_file: Job description file (optional if jd_text provided)
    - application_id: Link to application (optional)

    Returns:
    {
        "analysis_id": 1,
        "ats_score": 75,
        "matched_keywords": [...],
        "missing_keywords": [...],
        "suggestions": [...],
        "cv_file_path": "..."
    }
    """
    try:
        # Check if CV file is present
        if 'cv_file' not in request.files:
            return jsonify({"error": "cv_file is required"}), 400

        cv_file = request.files['cv_file']

        if cv_file.filename == '':
            return jsonify({"error": "No file selected"}), 400

        if not allowed_file(cv_file.filename):
            return jsonify({"error": "Invalid file type. Only PDF and DOCX allowed"}), 400

        # Check file size
        cv_file.seek(0, os.SEEK_END)
        file_size = cv_file.tell()
        cv_file.seek(0)

        max_size = current_app.config['MAX_CONTENT_LENGTH']
        if file_size > max_size:
            return jsonify({"error": f"File too large. Maximum size is {max_size // (1024*1024)}MB"}), 400

        # Get job description
        jd_text = request.form.get('jd_text', '').strip()
        jd_file = request.files.get('jd_file')
        application_id = request.form.get('application_id')

        if not jd_text and not jd_file:
            return jsonify({"error": "Either jd_text or jd_file is required"}), 400

        # Verify application if provided
        if application_id:
            app = db.query_one(
                "SELECT id FROM applications WHERE id = ? AND user_id = ?",
                (application_id, request.user_id)
            )
            if not app:
                return jsonify({"error": "Application not found"}), 404

        # Save CV file
        timestamp = datetime.now().strftime('%Ym%d_%H%M%S')
        original_filename = secure_filename(cv_file.filename)
        filename = f"{request.user_id}_{timestamp}_{original_filename}"

        upload_folder = current_app.config['UPLOAD_FOLDER']
        os.makedirs(upload_folder, exist_ok=True)

        cv_file_path = os.path.join(upload_folder, filename)
        cv_file.save(cv_file_path)

        # Extract text from CV
        try:
            if cv_file_path.endswith('.pdf'):
                cv_text = extract_text_from_pdf(cv_file_path)
            elif cv_file_path.endswith('.docx') or cv_file_path.endswith('.doc'):
                cv_text = extract_text_from_docx(cv_file_path)
            else:
                return jsonify({"error": "Unsupported file format"}), 400
        except Exception as e:
            # Clean up file
            if os.path.exists(cv_file_path):
                os.remove(cv_file_path)
            return jsonify({"error": f"Failed to extract text from CV: {str(e)}"}), 500

        # Extract text from JD
        if jd_file and jd_file.filename:
            # Save temporarily
            jd_temp_path = os.path.join(upload_folder, f"temp_jd_{timestamp}.txt")
            jd_file.save(jd_temp_path)

            try:
                if jd_temp_path.endswith('.pdf'):
                    jd_text = extract_text_from_pdf(jd_temp_path)
                elif jd_temp_path.endswith('.docx') or jd_temp_path.endswith('.doc'):
                    jd_text = extract_text_from_docx(jd_temp_path)
                else:
                    with open(jd_temp_path, 'r', encoding='utf-8') as f:
                        jd_text = f.read()
            finally:
                # Clean up temp file
                if os.path.exists(jd_temp_path):
                    os.remove(jd_temp_path)

        if not jd_text or len(jd_text.strip()) < 50:
            if os.path.exists(cv_file_path):
                os.remove(cv_file_path)
            return jsonify({"error": "Job description is too short (minimum 50 characters)"}), 400
    
```

```

# Extract keywords
cv_keywords = extract_keywords(cv_text)
jd_keywords = extract_keywords(jd_text)

# Calculate matched and missing keywords
matched_keywords = list(set(cv_keywords) & set(jd_keywords))
missing_keywords = list(set(jd_keywords) - set(cv_keywords))

# Calculate ATS score
ats_score = calculate_ats_score(cv_keywords, jd_keywords)

# Generate suggestions
suggestions = generate_suggestions(matched_keywords, missing_keywords)

# Insert into database
db.execute(
    """
    INSERT INTO cv_analyses
    (user_id, application_id, cv_filename, cv_file_path, job_description,
     ats_score, matched_keywords, missing_keywords, suggestions, created_at)
    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, datetime('now'))
    """,
    (
        request.user_id,
        application_id if application_id else None,
        original_filename,
        cv_file_path,
        jd_text,
        ats_score,
        json.dumps(matched_keywords),
        json.dumps(missing_keywords),
        json.dumps(suggestions)
    )
)
db.commit()

# Fetch created analysis
analysis = db.query_one(
    """
    SELECT id, ats_score, matched_keywords, missing_keywords, suggestions,
           cv_filename, cv_file_path, created_at
    FROM cv_analyses
    WHERE user_id = ?
    ORDER BY id DESC
    LIMIT 1
    """,
    (request.user_id,)
)

# Parse JSON fields
result = dict(analysis)
result['matched_keywords'] = json.loads(result['matched_keywords']) if result['matched_keywords'] else []
result['missing_keywords'] = json.loads(result['missing_keywords']) if result['missing_keywords'] else []
result['suggestions'] = json.loads(result['suggestions']) if result['suggestions'] else []

return jsonify({
    "analysis_id": result['id'],
    "ats_score": result['ats_score'],
    "matched_keywords": result['matched_keywords'],
    "missing_keywords": result['missing_keywords'],
    "suggestions": result['suggestions'],
    "cv_filename": result['cv_filename'],
    "cv_file_path": result['cv_file_path'],
    "created_at": result['created_at']
}), 201

except Exception as e:
    return jsonify({"error": str(e)}), 500

@cv_matcher_bp.route('/history', methods=['GET'])
@require_auth
def get_cv_history():
    """
    GET /api/v1/cv/history

    Get CV analysis history (protected)

    Returns:
    [
        {
            "id": 1,
            "ats_score": 75,
            "cv_filename": "resume.pdf",
            "created_at": "2025-01-15T10:00:00",
            "application_id": 2
        },
        ...
    ]
    """
    try:
        analyses = db.query(
            """
            SELECT id, ats_score, cv_filename, created_at, application_id
            FROM cv_analyses
            WHERE user_id = ?
            ORDER BY created_at DESC
            """,
            (request.user_id,)
        )

        return jsonify(analyses), 200

    except Exception as e:
        return jsonify({"error": str(e)}), 500

@cv_matcher_bp.route('/analysis/<int:analysis_id>', methods=['GET'])
@require_auth
def get_cv_analysis(analysis_id):
    """
    GET /api/v1/cv/analysis/<id>

    Get single CV analysis details (protected)

    Returns:
    {
        "id": 1,
        "ats_score": 75,
        "matched_keywords": [...],
        "missing_keywords": [...],
        "suggestions": [...],
        "cv_filename": "...",
        "job_description": "...",
    }
    """

```

```

        "created_at": "...")
    """
    try:
        analysis = db.query_one(
            """
            SELECT *
            FROM cv_analyses
            WHERE id = ? AND user_id = ?
            """,
            (analysis_id, request.user_id)
        )

        if not analysis:
            return jsonify({"error": "Analysis not found"}), 404

        # Parse JSON fields
        result = dict(analysis)
        result['matched_keywords'] = json.loads(result['matched_keywords']) if result['matched_keywords'] else []
        result['missing_keywords'] = json.loads(result['missing_keywords']) if result['missing_keywords'] else []
        result['suggestions'] = json.loads(result['suggestions']) if result['suggestions'] else []

        return jsonify(result), 200

    except Exception as e:
        return jsonify({"error": str(e)}), 500

7.
from flask import Blueprint, request, jsonify
from backend.database.db import db
from backend.utils.decorators import require_auth
from backend.services.streak_service import update_user_streak
from backend.services.notification_service import create_quest_completion_notification
from datetime import date, timedelta
import json
import os
import random

goals_bp = Blueprint('goals', __name__)

def get_streak_level(total_points: int) -> tuple:
    """
    Get streak level and next milestone based on total points

    Returns:
    (level_number, level_name, next_milestone)
    """
    if total_points < 100:
        return 1, "Getting Started", 100
    elif total_points < 300:
        return 2, "Momentum Builder", 300
    elif total_points < 600:
        return 3, "Consistent Achiever", 600
    else:
        return 4, "Job Hunt Pro", None

@goals_bp.route('/current', methods=['GET'])
@require_auth
def get_current_goal():
    """
    GET /api/v1/goals/current

    Get current week's goal (protected)

    Returns:
    {
        "goal": {
            "week_start": "2025-01-20",
            "applications_goal": 5,
            "applications_current": 2,
            "outreach_goal": 3,
            "outreach_current": 1,
            ...
        },
        "applications_percentage": 40.0,
        "outreach_percentage": 33.3,
        "days_remaining": 5
    }
    """
    try:
        # Determine week_start (Monday of current week)
        today = date.today()
        week_start = today - timedelta(days=today.weekday())

        # Fetch goal for current week
        goal = db.query_one(
            """
            SELECT *
            FROM goals
            WHERE user_id = ? AND week_start = ?
            """,
            (request.user_id, week_start.isoformat())
        )

        # If no goal exists, create one with default values
        if not goal:
            db.execute(
                """
                INSERT INTO goals
                (user_id, week_start, applications_goal, applications_current,
                 outreach_goal, outreach_current, created_at, updated_at)
                VALUES (?, ?, 5, 0, 3, 0, date('now'), NULL)
                """,
                (request.user_id, week_start.isoformat())
            )
            db.commit()

        goal = db.query_one(
            "SELECT * FROM goals WHERE user_id = ? AND week_start = ?",
            (request.user_id, week_start.isoformat())
        )

        # Calculate percentages
        applications_pct = (goal['applications_current'] / goal['applications_goal'] * 100) \
            if goal['applications_goal'] > 0 else 0
        outreach_pct = (goal['outreach_current'] / goal['outreach_goal'] * 100) \
            if goal['outreach_goal'] > 0 else 0

        # Calculate days remaining in week (0-6, where 0 = Sunday)
        days_remaining = 6 - today.weekday()

        return jsonify({
            "goal": goal,

```

```

        "applications_percentage": round(applications_pct, 1),
        "outreach_percentage": round(outreach_pct, 1),
        "days_remaining": days_remaining
    )), 200

except Exception as e:
    return jsonify({"error": str(e)}), 500

@ggoals_bp.route('/update', methods=['POST'])
@require_auth
def update_goals():
    """
    POST /api/v1/goals/update

    Update weekly goals (protected)

    Can only update once per week

    Request body:
    {
        "applications_goal": 7,
        "outreach_goal": 5
    }

    Returns:
    Updated goal object
    """
    try:
        data = request.get_json()

        if not data:
            return jsonify({"error": "No data provided"}), 400

        applications_goal = data.get('applications_goal')
        outreach_goal = data.get('outreach_goal')

        if applications_goal is None or outreach_goal is None:
            return jsonify({"error": "applications_goal and outreach_goal are required"}), 400

        # Validate goals are positive
        if applications_goal < 1 or outreach_goal < 1:
            return jsonify({"error": "Goals must be positive integers"}), 400

        # Get current week
        today = date.today()
        week_start = today - timedelta(days=today.weekday())

        # Fetch existing goal
        goal = db.query_one(
            "SELECT * FROM goals WHERE user_id = ? AND week_start = ?",
            (request.user_id, week_start.isoformat())
        )

        if not goal:
            return jsonify({"error": "No goal found for current week"}), 404

        # Check if already updated this week
        if goal['updated_at']:
            # Parse updated_at (handle both date and datetime formats)
            # Split by space to get date part: '2025-11-25 22:55:02' -> '2025-11-25'
            updated_str = str(goal['updated_at']).split()[0]
            updated_date = date.fromisoformat(updated_str)
            week_start_date = date.fromisoformat(goal['week_start'])

            # If updated_at is within the same week, don't allow another update
            if updated_date >= week_start_date:
                return jsonify({
                    "error": "Goals can only be updated once per week"
                }), 400

        # Update goals
        db.execute(
            """
            UPDATE goals
            SET applications_goal = ?, outreach_goal = ?, updated_at = date('now')
            WHERE user_id = ? AND week_start = ?
            """
            (applications_goal, outreach_goal, request.user_id, week_start.isoformat())
        )
        db.commit()

        # Fetch updated goal
        updated_goal = db.query_one(
            "SELECT * FROM goals WHERE user_id = ? AND week_start = ?",
            (request.user_id, week_start.isoformat())
        )

        return jsonify(updated_goal), 200

    except Exception as e:
        return jsonify({"error": str(e)}), 500

@ggoals_bp.route('/streak', methods=['GET'])
@require_auth
def get_streak():
    """
    GET /api/v1/goals/streak

    Get current streak information (protected)

    Returns:
    {
        "current_streak": 5,
        "longest_streak": 12,
        "last_activity_date": "2025-01-15",
        "total_points": 450,
        "level": 3,
        "level_name": "Consistent Achiever",
        "next_milestone": 600
    }
    """
    try:
        # Fetch streak
        streak = db.query_one(
            "SELECT * FROM streaks WHERE user_id = ?",
            (request.user_id,)
        )

        if not streak:
            return jsonify({"error": "Streak not found"}), 404

        # Calculate level and next milestone

```



```

        level_number, level_name, next_milestone = get_streak_level(streak['total_points'])

    return jsonify({
        "current_streak": streak['current_streak'],
        "longest_streak": streak['longest_streak'],
        "last_activity_date": streak['last_activity_date'],
        "total_points": streak['total_points'],
        "level": level_number,
        "level_name": level_name,
        "next_milestone": next_milestone
    }), 200

except Exception as e:
    return jsonify({"error": str(e)}), 500

@goals_bp.route('/micro-quests', methods=['GET'])
@require_auth
def get_micro_quests():
    """
    GET /api/v1/goals/micro-quests

    Get available micro-quests (3 random, not completed) (protected)

    Returns:
    [
        {
            "id": "mq-1",
            "title": "Update Your LinkedIn Profile",
            "description": "...",
            "category": "profile_optimization",
            "points": 20
        },
        ...
    ]
    """
    try:
        # Load micro-quests from JSON
        quests_path = os.path.join(os.path.dirname(__file__), '..', 'data', 'micro_quests.json')
        with open(quests_path, 'r') as f:
            quests_data = json.load(f)

        all_quests = quests_data['quests']

        # Fetch completed quest IDs
        completed = db.query(
            "SELECT quest_id FROM user_quests WHERE user_id = ?",
            (request.user_id,)
        )
        completed_ids = [row['quest_id'] for row in completed]

        # Filter out completed quests
        available_quests = [q for q in all_quests if q['id'] not in completed_ids]

        # Return 3 random quests if available
        if len(available_quests) > 3:
            selected_quests = random.sample(available_quests, 3)
        else:
            selected_quests = available_quests

        return jsonify(selected_quests), 200

    except Exception as e:
        return jsonify({"error": str(e)}), 500

@goals_bp.route('/micro-quests/<quest_id>/complete', methods=['POST'])
@require_auth
def complete_micro_quest(quest_id):
    """
    POST /api/v1/goals/micro-quests/<quest_id>/complete

    Mark micro-quest as completed (protected)

    Returns:
    {
        "message": "Quest completed!",
        "points_earned": 20,
        "new_total_points": 470,
        "quest": {...}
    }
    """
    try:
        # Check if already completed
        existing = db.query_one(
            "SELECT id FROM user_quests WHERE user_id = ? AND quest_id = ?",
            (request.user_id, quest_id)
        )

        if existing:
            return jsonify({"error": "Quest already completed"}), 400

        # Load quest details
        quests_path = os.path.join(os.path.dirname(__file__), '..', 'data', 'micro_quests.json')
        with open(quests_path, 'r') as f:
            quests_data = json.load(f)

        # Find the quest
        quest = None
        for q in quests_data['quests']:
            if q['id'] == quest_id:
                quest = q
                break

        if not quest:
            return jsonify({"error": "Quest not found"}), 404

        # Insert completion record
        db.execute(
            """
            INSERT INTO user_quests (user_id, quest_id, completed_at)
            VALUES (?, ?, datetime('now'))
            """
            (request.user_id, quest_id)
        )
        db.commit()

        # Update streak with quest points
        quest_points = quest.get('points', 10)
        streak_result = update_user_streak(request.user_id, points=quest_points)

        # Create completion notification
        create_quest_completion_notification(
            request.user_id,

```

```

        quest['title'],
        quest_points
    )

    return jsonify({
        "message": "Quest completed!",
        "points_earned": quest_points,
        "new_total_points": streak_result['total_points'],
        "quest": quest
    }), 200

except Exception as e:
    return jsonify({"error": str(e)}), 500

8.
from flask import Blueprint, request, jsonify
from backend.database.db import db
from backend.utils.decorators import require_auth
from datetime import date, timedelta

onboarding_bp = Blueprint('onboarding', __name__)

VALID_FEELINGS = [
    'Excited and ready',
    'Overwhelmed but motivated',
    'Frustrated and stuck',
    'Just getting started'
]

@onboarding_bp.route('', methods=['POST'])
@require_auth
def create_onboarding():
    """
    POST /api/v1/onboarding

    Create onboarding data for user (protected)

    Request body:
    {
        "current_feeling": "Excited and ready",
        "dream_milestone": "Become a Senior Developer at FAANG",
        "weekly_application_goal": 7,
        "weekly_outreach_goal": 5,
        "companies": [
            {"name": "Google", "website": "https://google.com"},
            {"name": "Microsoft", "website": "https://microsoft.com"}
        ]
    }

    Returns:
    {
        "onboarding": {...},
        "goal": {...},
        "companies_created": 2
    }
    """
    try:
        data = request.get_json()

        if not data:
            return jsonify({"error": "No data provided"}), 400

        # Validate required fields
        if not data.get('current_feeling'):
            return jsonify({"error": "current_feeling is required"}), 400
        if not data.get('dream_milestone'):
            return jsonify({"error": "dream_milestone is required"}), 400

        current_feeling = data['current_feeling']
        dream_milestone = data['dream_milestone'].strip()
        weekly_application_goal = data.get('weekly_application_goal', 5)
        weekly_outreach_goal = data.get('weekly_outreach_goal', 3)
        companies = data.get('companies', [])

        # Validate current_feeling
        if current_feeling not in VALID_FEELINGS:
            return jsonify({
                "error": f"Invalid feeling. Must be one of: {' '.join(VALID_FEELINGS)}"
            }), 400

        # Validate dream_milestone length
        if len(dream_milestone) < 10:
            return jsonify({"error": "dream_milestone must be at least 10 characters"}), 400

        # Validate goals are positive
        if weekly_application_goal < 1 or weekly_outreach_goal < 1:
            return jsonify({"error": "Goals must be positive integers"}), 400

        # Check if onboarding already exists
        existing = db.query_one(
            "SELECT id FROM onboarding_data WHERE user_id = ?",
            (request.user_id,)
        )

        if existing:
            return jsonify({"error": "Onboarding already completed"}), 400

        # Insert onboarding data
        db.execute(
            """
            INSERT INTO onboarding_data
            (user_id, current_feeling, dream_milestone, completed_at)
            VALUES (?, ?, ?, datetime('now'))
            """
            (request.user_id, current_feeling, dream_milestone)
        )
        db.commit()

        # Fetch created onboarding
        onboarding = db.query_one(
            "SELECT * FROM onboarding_data WHERE user_id = ?",
            (request.user_id,)
        )

        # Compute week_start (Monday of current week)
        today = date.today()
        week_start = today - timedelta(days=today.weekday())

        # Insert initial goal row
        db.execute(
            """
            INSERT INTO goals
            (user_id, week_start, applications_goal, applications_current,
            outreach_goal, outreach_current, created_at, updated_at)
            """

```

```

VALUES (?, ?, ?, 0, ?, 0, date('now')), NULL)
"""
(request.user_id, week_start.isoformat(), weekly_application_goal, weekly_outreach_goal)
)
db.commit()

# Fetch created goal
goal = db.query_one(
    "SELECT * FROM goals WHERE user_id = ? AND week_start = ?",
    (request.user_id, week_start.isoformat())
)

# Create companies
companies_created = 0
for company_data in companies:
    if not company_data.get('name'):
        continue

    name = company_data['name'].strip()
    website = company_data.get('website', '').strip() or None

    # Check if company already exists for this user
    existing_company = db.query_one(
        "SELECT id FROM companies WHERE user_id = ? AND LOWER(name) = LOWER(?)",
        (request.user_id, name)
    )

    if not existing_company:
        db.execute(
            """
            INSERT INTO companies (user_id, name, website, created_at)
            VALUES (?, ?, ?, date('now'))
            """
            (request.user_id, name, website)
        )
        companies_created += 1

db.commit()

return jsonify({
    "onboarding": onboarding,
    "goal": goal,
    "companies_created": companies_created
}), 201

except Exception as e:
    return jsonify({"error": str(e)}), 500

@onboarding_bp.route('', methods=['GET'])
@require_auth
def get_onboarding():
    """
    GET /api/v1/onboarding

    Get onboarding data for current user (protected)

    Returns:
    {
        "completed": true,
        "current_feeling": "...",
        "dream_milestone": "...",
        "completed_at": "..."
    }

    OR

    {
        "completed": false
    }
    """
    try:
        onboarding = db.query_one(
            "SELECT * FROM onboarding_data WHERE user_id = ?",
            (request.user_id,)
        )

        if not onboarding:
            return jsonify({"completed": False}), 200

        return jsonify({
            "completed": True,
            "id": onboarding['id'],
            "current_feeling": onboarding['current_feeling'],
            "dream_milestone": onboarding['dream_milestone'],
            "completed_at": onboarding['completed_at']
        }), 200

    except Exception as e:
        return jsonify({"error": str(e)}), 500

```

```

9. from flask import Blueprint, request, jsonify
from backend.database.db import db
from backend.util.decorators import require_auth
from datetime import datetime

notifications_bp = Blueprint('notifications', __name__)

VALID_NOTIFICATION_TYPES = ['follow_up', 'goal_reminder', 'micro_quest', 'motivation', 'system']

@notifications_bp.route('', methods=['GET'])
@require_auth
def get_notifications():
    """
    GET /api/v1/notifications

    Get all notifications for user (protected)

    Query params:
    - unread: Filter unread only (true/false)

    Returns:
    {
        "notifications": [...],
        "unread_count": 5
    }
    """
    try:
        unread_only = request.args.get('unread', '').lower() == 'true'

        # Build query
        if unread_only:
            notifications = db.query(

```

```

        """
        SELECT *
        FROM notifications
        WHERE user_id = ? AND is_read = 0
        ORDER BY datetime(created_at) DESC
        """
        (request.user_id,
    )
    else:
        notifications = db.query(
            """
            SELECT *
            FROM notifications
            WHERE user_id = ?
            ORDER BY datetime(created_at) DESC
            """
            (request.user_id,
        )

    # Get unread count
    unread_count_row = db.query_one(
        "SELECT COUNT(*) AS cnt FROM notifications WHERE user_id = ? AND is_read = 0",
        (request.user_id,
    )
    unread_count = unread_count_row['cnt'] if unread_count_row else 0

    return jsonify({
        "notifications": notifications,
        "unread_count": unread_count
    }), 200

except Exception as e:
    return jsonify({"error": str(e)}), 500

@notifications_bp.route('/unread-count', methods=['GET'])
@require_auth
def get_unread_count():
    """
    GET /api/v1/notifications/unread-count

    Get count of unread notifications (protected)

    This endpoint is polled by frontend every 60 seconds

    Returns:
    {
        "unread_count": 5
    }
    """
    try:
        unread_count_row = db.query_one(
            "SELECT COUNT(*) AS cnt FROM notifications WHERE user_id = ? AND is_read = 0",
            (request.user_id,
        )

        unread_count = unread_count_row['cnt'] if unread_count_row else 0

        return jsonify({"unread_count": unread_count}), 200

    except Exception as e:
        return jsonify({"error": str(e)}), 500

@notifications_bp.route('/<int:notif_id>/read', methods=['PUT'])
@require_auth
def mark_notification_read(notif_id):
    """
    PUT /api/v1/notifications/<id>/read

    Mark notification as read (protected)

    Returns:
    {
        "message": "Notification marked as read",
        "notification": {...}
    }
    """
    try:
        # Verify notification belongs to user
        notif = db.query_one(
            "SELECT id, is_read FROM notifications WHERE id = ? AND user_id = ?",
            (notif_id, request.user_id)
        )

        if not notif:
            return jsonify({"error": "Notification not found"}), 404

        # Update to read
        db.execute(
            "UPDATE notifications SET is_read = 1 WHERE id = ?",
            (notif_id,
        )
        db.commit()

        # Fetch updated notification
        updated_notif = db.query_one(
            "SELECT * FROM notifications WHERE id = ?",
            (notif_id,
        )

        return jsonify({
            "message": "Notification marked as read",
            "notification": updated_notif
        }), 200

    except Exception as e:
        return jsonify({"error": str(e)}), 500

@notifications_bp.route('/read-all', methods=['PUT'])
@require_auth
def mark_all_read():
    """
    PUT /api/v1/notifications/read-all

    Mark all notifications as read (protected)

    Returns:
    {
        "message": "All notifications marked as read",
        "count": 10
    }
    """

```

```

try:
    # Count unread before update
    unread_count_row = db.query_one(
        "SELECT COUNT(*) AS cnt FROM notifications WHERE user_id = ? AND is_read = 0",
        (request.user_id,)
    )
    count = unread_count_row['cnt'] if unread_count_row else 0

    # Update all to read
    db.execute(
        "UPDATE notifications SET is_read = 1 WHERE user_id = ? AND is_read = 0",
        (request.user_id,)
    )
    db.commit()

    return jsonify({
        "message": "All notifications marked as read",
        "count": count
    }), 200

except Exception as e:
    return jsonify({"error": str(e)}), 500

@notifications_bp.route('/<int:notif_id>', methods=['DELETE'])
@require_auth
def delete_notification(notif_id):
    """
    DELETE /api/v1/notifications/<id>

    Delete notification (protected)

    Returns:
    {
        "message": "Notification deleted successfully"
    }
    """
    try:
        # Verify notification belongs to user
        notif = db.query_one(
            "SELECT id FROM notifications WHERE id = ? AND user_id = ?",
            (notif_id, request.user_id)
        )

        if not notif:
            return jsonify({"error": "Notification not found"}), 404

        # Delete notification
        db.execute(
            "DELETE FROM notifications WHERE id = ?",
            (notif_id,)
        )
        db.commit()

        return jsonify({"message": "Notification deleted successfully"}), 200

    except Exception as e:
        return jsonify({"error": str(e)}), 500

@notifications_bp.route('/clear-all', methods=['DELETE'])
@require_auth
def clear_all_notifications():
    """
    DELETE /api/v1/notifications/clear-all

    Delete all notifications for user (protected)

    Returns:
    {
        "message": "All notifications cleared",
        "count": 15
    }
    """
    try:
        # Count before delete
        count_row = db.query_one(
            "SELECT COUNT(*) AS cnt FROM notifications WHERE user_id = ?",
            (request.user_id,)
        )
        count = count_row['cnt'] if count_row else 0

        # Delete all notifications
        db.execute(
            "DELETE FROM notifications WHERE user_id = ?",
            (request.user_id,)
        )
        db.commit()

        return jsonify({
            "message": "All notifications cleared",
            "count": count
        }), 200

    except Exception as e:
        return jsonify({"error": str(e)}), 500

10.rom flask import Blueprint, request, jsonify
from backend.database.db import db
from backend.utils.decorators import require_auth
from backend.utils.validators import validate_date
from backend.services.streak_service import update_user_streak
from backend.services.notification_service import create_follow_up_notification
from datetime import date, timedelta
import json
import os

outreach_bp = Blueprint('outreach', __name__)

VALID_CHANNELS = ['email', 'linkedin']

@outreach_bp.route('/templates/generate', methods=['POST'])
@require_auth
def generate_template():
    """
    POST /api/v1/outreach/templates/generate

    Generate personalized outreach template (protected)

    Request body:
    {
        "contact_id": 1,
        "company_id": 1,
        "application_id": 2, # optional
    }

```

```

        "template_type": "cold_outreach"
    }

Returns:
{
    "subject": "...",
    "body": "...",
    "editing_tips": "..."
}
"""
try:
    data = request.get_json()

    if not data:
        return jsonify({"error": "No data provided"}), 400

    contact_id = data.get('contact_id')
    company_id = data.get('company_id')
    application_id = data.get('application_id')
    template_type = data.get('template_type', 'cold_outreach')

    if not contact_id or not company_id:
        return jsonify({"error": "contact_id and company_id are required"}), 400

    # Fetch contact
    contact = db.query_one(
        """
        SELECT c.*, comp.user_id
        FROM contacts c
        JOIN companies comp ON c.company_id = comp.id
        WHERE c.id = ?
        """,
        (contact_id,)
    )

    if not contact or contact['user_id'] != request.user_id:
        return jsonify({"error": "Contact not found"}), 404

    # Fetch company
    company = db.query_one(
        "SELECT * FROM companies WHERE id = ? AND user_id = ?",
        (company_id, request.user_id)
    )

    if not company:
        return jsonify({"error": "Company not found"}), 404

    # Fetch user
    user = db.query_one(
        "SELECT * FROM users WHERE id = ?",
        (request.user_id,)
    )

    # Fetch onboarding data for dream_milestone
    onboarding = db.query_one(
        "SELECT dream_milestone FROM onboarding_data WHERE user_id = ?",
        (request.user_id,)
    )

    dream_milestone = onboarding['dream_milestone'] if onboarding else "achieving my career goals"

    # Fetch application if provided
    application = None
    if application_id:
        application = db.query_one(
            "SELECT * FROM applications WHERE id = ? AND user_id = ?",
            (application_id, request.user_id)
        )

    # Load templates
    templates_path = os.path.join(os.path.dirname(__file__), '..', 'data', 'templates.json')
    with open(templates_path, 'r') as f:
        templates_data = json.load(f)

    # Find template
    template = None
    for t in templates_data['templates']:
        if t['type'] == template_type:
            template = t
            break

    if not template:
        return jsonify({"error": "Template not found"}), 404

    # Replace tokens
    subject = template['subject']
    body = template['body']

    replacements = {
        '{contact_name}': contact['name'] or 'there',
        '{company_name}': company['name'],
        '{user_name}': user['name'],
        '{dream_milestone}': dream_milestone,
        '{industry}': company['industry'] or 'your industry',
        '{job_title}': application['job_title'] if application else 'the position',
        '{applied_date}': application['applied_date'] if application else 'recently'
    }

    for token, value in replacements.items():
        subject = subject.replace(token, value)
        body = body.replace(token, value)

    return jsonify({
        "subject": subject,
        "body": body,
        "editing_tips": template.get('editing_tips', '')
    }), 200

except Exception as e:
    return jsonify({"error": str(e)}), 500

@outreach_bp.route('', methods=['POST'])
@require_auth
def create_outreach():
    """
    POST /api/v1/outreach

    Create outreach activity (protected)

    Request body:
    {
        "application_id": 1, # OR company_id (exactly one required)
        "company_id": 1,
    }

```

```

        "contact_id": 1,
        "channel": "email",
        "message": "...",
        "sent_date": "2025-01-15",
        "follow_up_date": "2025-01-22" # optional
    )

Returns:
    Created outreach object
"""
try:
    data = request.get_json()

    if not data:
        return jsonify({"error": "No data provided"}), 400

    application_id = data.get('application_id')
    company_id = data.get('company_id')
    contact_id = data.get('contact_id')
    channel = data.get('channel')
    message = data.get('message', '').strip()
    sent_date = data.get('sent_date')
    follow_up_date = data.get('follow_up_date')

    # Validate required fields
    if not contact_id:
        return jsonify({"error": "contact_id is required"}), 400
    if not channel:
        return jsonify({"error": "channel is required"}), 400
    if not message:
        return jsonify({"error": "message is required"}), 400
    if not sent_date:
        return jsonify({"error": "sent_date is required"}), 400

    # Validate XOR: exactly one of application_id or company_id
    if (application_id is None and company_id is None) or \
        (application_id is not None and company_id is not None):
        return jsonify({
            "error": "Must provide exactly ONE of application_id or company_id"
        }), 400

    # Validate channel
    if channel not in VALID_CHANNELS:
        return jsonify({
            "error": f"Invalid channel. Must be one of: {'', '.join(VALID_CHANNELS)}"
        }), 400

    # Validate dates
    if not validate_date(sent_date):
        return jsonify({"error": "Invalid sent_date format. Use YYYY-MM-DD"}), 400

    if follow_up_date and not validate_date(follow_up_date):
        return jsonify({"error": "Invalid follow_up_date format. Use YYYY-MM-DD"}), 400

    # Validate message length
    if len(message) < 10:
        return jsonify({"error": "Message must be at least 10 characters"}), 400

    # Verify contact belongs to user
    contact = db.query_one(
        """
        SELECT c.id, comp.user_id
        FROM contacts c
        JOIN companies comp ON c.company_id = comp.id
        WHERE c.id = ?
        """,
        (contact_id,)
    )

    if not contact or contact['user_id'] != request.user_id:
        return jsonify({"error": "Contact not found"}), 404

    # Verify application or company belongs to user
    if application_id:
        app = db.query_one(
            "SELECT id FROM applications WHERE id = ? AND user_id = ?",
            (application_id, request.user_id)
        )
        if not app:
            return jsonify({"error": "Application not found"}), 404

    if company_id:
        comp = db.query_one(
            "SELECT id FROM companies WHERE id = ? AND user_id = ?",
            (company_id, request.user_id)
        )
        if not comp:
            return jsonify({"error": "Company not found"}), 404

    # Insert outreach
    db.execute(
        """
        INSERT INTO outreach_activities
        (user_id, application_id, company_id, contact_id, channel,
         message_template, sent_date, follow_up_date, status, created_at)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, 'Sent', datetime('now'))
        """,
        (request.user_id, application_id, company_id, contact_id,
         channel, message, sent_date, follow_up_date)
    )
    db.commit()

    # Fetch created outreach
    outreach = db.query_one(
        """
        SELECT o.*, c.name AS contact_name, c.email AS contact_email
        FROM outreach_activities o
        LEFT JOIN contacts c ON o.contact_id = c.id
        WHERE o.user_id = ?
        ORDER BY o.id DESC
        LIMIT 1
        """,
        (request.user_id,)
    )

    # Update goal for current week
    today = date.today()
    week_start = today - timedelta(days=today.weekday())

    db.execute(
        """
        UPDATE goals
        SET outreach_current = outreach_current + 1,
            updated_at = date('now')
        """
    )

```

```

        WHERE user_id = ? AND week_start = ?
        """
        (request.user_id, week_start.isoformat())
    )
    db.commit()

    # Update streak
    update_user_streak(request.user_id, points=10)

    # Create follow-up notification if follow_up_date provided
    if follow_up_date and application_id:
        # Calculate days until follow-up
        follow_date = date.fromisoformat(follow_up_date)
        days_ahead = (follow_date - date.today()).days
        if days_ahead > 0:
            create_follow_up_notification(request.user_id, application_id, days_ahead)

    return jsonify(outreach), 201

except Exception as e:
    return jsonify({"error": str(e)}), 500

@outreach_bp.route('', methods=['GET'])
@require_auth
def get_outreach():
    """
    GET /api/v1/outreach

    Get outreach activities (protected)

    Query params:
    - application_id: Filter by application
    - company_id: Filter by company

    Returns:
    [{"id": 1, "channel": "email", ...}, ...]
    """
    try:
        application_id = request.args.get('application_id')
        company_id = request.args.get('company_id')

        if application_id:
            outreach_list = db.query(
                """
                SELECT o.*, c.name AS contact_name, c.email AS contact_email,
                       comp.name AS company_name
                FROM outreach_activities o
                LEFT JOIN contacts c ON o.contact_id = c.id
                LEFT JOIN companies comp ON o.company_id = comp.id
                WHERE o.user_id = ? AND o.application_id = ?
                ORDER BY o.sent_date DESC
                """
                , (request.user_id, application_id)
            )
        elif company_id:
            outreach_list = db.query(
                """
                SELECT o.*, c.name AS contact_name, c.email AS contact_email,
                       comp.name AS company_name
                FROM outreach_activities o
                LEFT JOIN contacts c ON o.contact_id = c.id
                LEFT JOIN companies comp ON o.company_id = comp.id
                WHERE o.user_id = ? AND o.company_id = ?
                ORDER BY o.sent_date DESC
                """
                , (request.user_id, company_id)
            )
        else:
            outreach_list = db.query(
                """
                SELECT o.*, c.name AS contact_name, c.email AS contact_email,
                       comp.name AS company_name
                FROM outreach_activities o
                LEFT JOIN contacts c ON o.contact_id = c.id
                LEFT JOIN companies comp ON o.company_id = comp.id OR o.application_id IN (
                    SELECT id FROM applications WHERE company_id = comp.id
                )
                WHERE o.user_id = ?
                ORDER BY o.sent_date DESC
                """
                , (request.user_id,)
            )

        return jsonify(outreach_list), 200

    except Exception as e:
        return jsonify({"error": str(e)}), 500

@outreach_bp.route('/<int:outreach_id>', methods=['PUT'])
@require_auth
def update_outreach(outreach_id):
    """
    PUT /api/v1/outreach/<id>

    Update outreach status (protected)

    Request body:
    {
        "status": "Responded"
    }

    Returns:
    Updated outreach object
    """
    try:
        data = request.get_json()

        if not data or 'status' not in data:
            return jsonify({"error": "status is required"}), 400

        status = data['status']

        if status not in ['Sent', 'Responded', 'No Response']:
            return jsonify({"error": "Invalid status"}), 400

        # Verify outreach belongs to user
        outreach = db.query_one(
            "SELECT id FROM outreach_activities WHERE id = ? AND user_id = ?",
            (outreach_id, request.user_id)
        )

        if not outreach:

```



```

        return jsonify({"error": "Outreach not found"}), 404

    # Update status
    db.execute(
        "UPDATE outreach_activities SET status = ? WHERE id = ?",
        (status, outreach_id)
    )
    db.commit()

    # Fetch updated outreach
    updated = db.query_one(
        """
        SELECT o.*, c.name AS contact_name, c.email AS contact_email
        FROM outreach_activities o
        LEFT JOIN contacts c ON o.contact_id = c.id
        WHERE o.id = ?
        """,
        (outreach_id,)
    )

    return jsonify(updated), 200

except Exception as e:
    return jsonify({"error": str(e)}), 500

```